

Assignment 1

Sanskruti Dahiphale – 123B1B007

1. Problem Statement

You are working as a data analyst for a financial institution that handles high-precision transactions involving large, multi-digit monetary values. Recently, the institution needs to calculate the product of two exceptionally large numbers (each over 15 digits) representing aggregated transaction amounts for quarterly reports. Due to the size of these values, traditional multiplication methods are computationally inefficient.

Implement the algorithm to perform high-precision multiplication of these large integers accurately and efficiently. Test your implementation with multiple large-number pairs to ensure reliability and precision in financial reporting.

2. Prerequisite

- Knowledge of **Divide and Conquer Strategy**
- Basic understanding of recursion
- Knowledge of time complexity analysis

3. Objectives

- To understand and implement the **Divide and Conquer strategy**.
- To solve real-world problems involving exceptionally large datasets that exceed standard data type limits.

4. Course Outcome Mapping

CO1: Apply divide and conquer strategy.

CO2: Analyze time complexity of recursive algorithms.

CO3: Implement efficient algorithms for large numbers.

5. Theory

Divide & Conquer Strategy :

Divide and Conquer is an algorithm design technique that:

1. **Divide** → Break the problem into smaller subproblems.
2. **Conquer** → Solve subproblems recursively.
3. **Combine** → Merge the solutions to get the final result.

This strategy reduces complexity by solving smaller instances of the same problem.

Example of Divide & Conquer Strategy

Merge Sort is a classic example:

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the sorted halves.

Time Complexity of Merge Sort:

$$\begin{aligned} T(n) &= 2T(n/2) + O(n) \\ &= O(n \log n) \end{aligned}$$

Algorithm of Divide & Conquer Strategy (High-Precision Multiplication)

Regular Multiplication

Traditional multiplication performs digit-by-digit multiplication.

Time Complexity: $O(n^2)$

This becomes inefficient for very large numbers.

Karatsuba Algorithm

The Karatsuba algorithm (developed by **Anatoly Karatsuba**) improves multiplication using Divide and Conquer.

If: $X = X_1 \cdot 10^m + X_0$

$Y = Y_1 \cdot 10^m + Y_0$

Then:

$$XY = z_0 \cdot 10^{2m} + (z_1 - z_0 - z_2) \cdot 10^m + z_2$$

Where:

- $Z_0 = X_1 Y_1$
- $Z_1 = (X_1 + X_0)(Y_1 + Y_0)$
- $Z_2 = X_0 Y_0$

It reduces 4 multiplications to 3 recursive multiplications.

Time Complexity with Explanation

Recurrence relation: $T(n) = 3T(n/2) + O(n)$

Using Master's Theorem:

$$T(n) = O(n^{\log 3})$$

$$T(n) \approx O(n^{1.585})$$

Since $1.585 < 2$, Karatsuba is faster than traditional $O(n^2)$ multiplication for large numbers.

Algorithm: Karatsuba Multiplication

Input: Two integers X and Y

Output: Product of X and Y

1. If X and Y are single-digit numbers:
 - Return $X \times Y$. (Base Case)
2. Determine n = maximum number of digits of X and Y.
3. Compute $m = n // 2$.
4. Split the numbers:
 - $X_1 = X // 10^m$ ◦ $X_0 = X \% 10^m$ ◦
 $Y_1 = Y // 10^m$ ◦ $Y_0 = Y \% 10^m$
5. Recursively compute:

- $z_0 = \text{karatsuba}(X_1, Y_1)$ ○ $z_2 = \text{karatsuba}(X_0, Y_0)$ ○ $z_1 = \text{karatsuba}(X_1 + X_0, Y_1 + Y_0)$ 6.

Combine:

- $\text{Result} = z_0 \times 10^{2m} + (z_1 - z_0 - z_2) \times 10^m + z_2$

7. Return Result.

Example of Karatsuba:

Step 1: Split the Numbers

$$X = 1234$$

$$Y = 5678$$

$$n = 4 \quad m$$

$$= 2$$

Split each number into two halves:

$$X_1 = 12$$

$$X_0 = 34$$

$$Y_1 = 56$$

$$Y_0 = 78$$

Step 2: Compute Three Multiplications

$$Z_0 = X_0 \times Y_0$$

$$Z_0 = 34 \times 78$$

$$Z_0 = 2652$$

$$Z_2 = X_1 \times Y_1 \quad Z_2$$

$$= 12 \times 56$$

$$Z_2 = 672$$

$$Z_1 = (X_1 + X_0) \times (Y_1 + Y_0)$$

$$Z_1 = (12 + 34) \times (56 + 78)$$

$$Z_1 = 46 \times 134$$

$$Z_1 = 6164$$

Step 3: Compute the Middle Term Middle

$$= Z_1 - Z_2 - Z_0$$

$$\text{Middle} = 6164 - 672 - 2652 \quad \text{Middle} = 2840$$

Step 4: Combine Results

$$\text{Result} = Z_2 \times 10^{(2m)} + \text{Middle} \times 10^m + Z_0$$

Substitute values:

$$\text{Result} = 672 \times 10^4 + 2840 \times 10^2 + 2652$$

$$\text{Result} = 6,720,000 + 284,000 + 2,652 \quad \text{Result}$$

$$= \mathbf{7,006,652}$$

6. Implementation

```
def karatsuba(X,Y):    if
X < 10 and Y < 10:
    return X*Y

    n = max(len(str(X)), len(str(Y)))
    m = n // 2

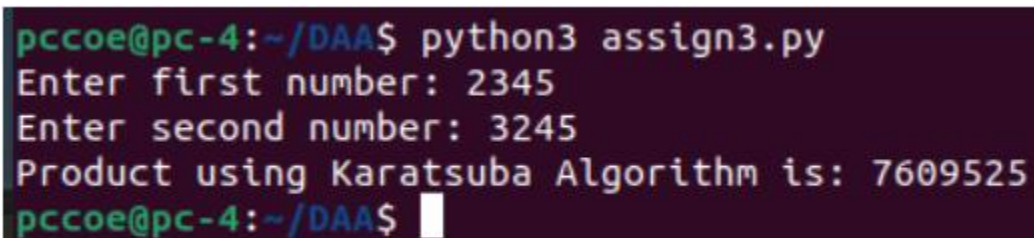
    X1 = X // 10**m
    X0 = X % 10**m
    Y1 = Y // 10**m
    Y0 = Y % 10**m

    # Recursive function call for multiplication
    z0 = karatsuba(X1,Y1)    z1 =
karatsuba(X1+X0, Y1+Y0)    z2 =
karatsuba(X0, Y0)

    ans = z0*10**(2*m) + (z1-z0-z2)*10**m + z2
    return ans

a = int(input("Enter the first number: ")) b =
int(input("Enter the second number: "))
result = karatsuba(a,b) print(result)
```

7. Sample Input & Output



```
pccoe@pc-4:~/DAA$ python3 assign3.py
Enter first number: 2345
Enter second number: 3245
Product using Karatsuba Algorithm is: 7609525
pccoe@pc-4:~/DAA$
```

8. Conclusion

Hence, we have studied the Divide and Conquer strategy with an example. The Karatsuba algorithm efficiently multiplies large integers by reducing the number of recursive multiplications. Compared to the traditional $O(n^2)$ method, it improves performance to $O(n^{\{1.585\}})$.

Therefore, it is highly suitable for high-precision financial calculations involving very large transaction values